Stocker – A Cloud based stock trading application using Flask and AWS

Introduction

With the rapid growth of digital finance, traditional trading platforms often struggle to provide real-time execution, secure portfolio management, and seamless user experiences. To address these challenges, **Stocker** was developed as a next-generation, cloud-native stock trading application.

Built using **Flask** for backend development, hosted on **AWS EC2**, and integrated with **DynamoDB** for scalable data management, Stocker enables users to perform secure, real-time trades, manage portfolios, and access market insights. The system emphasizes speed, reliability, and security, ensuring investors can trade confidently in today's fast-moving markets.

Problem Statement

Conventional trading systems often suffer from latency, limited scalability, and weak security measures. These limitations prevent users from executing trades quickly and managing portfolios effectively. A modern, cloud-based solution is required to deliver **real-time trading, secure authentication, and intuitive portfolio tracking**.

Objectives

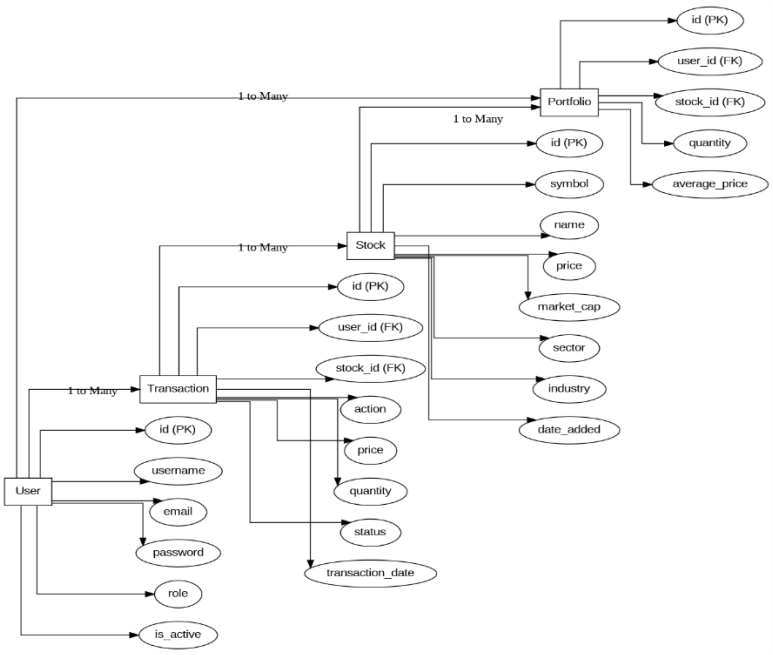
- Develop a secure, cloud-native stock trading platform.
- Provide **real-time trade execution** with minimal latency.
- Enable **portfolio tracking and analysis** through interactive dashboards.
- Ensure **robust authentication and account security**.
- Deliver a smooth, user-friendly trading experience.

System Overview

Stocker is designed as a **cloud-based application** that allows users to register, authenticate securely, execute trades instantly, and monitor their portfolios. The backend uses Flask, DynamoDB stores user and trade data, and EC2 hosts the application. The frontend is styled with modern CSS for an interactive experience.

This architecture ensures **scalability, high availability, and secure data management**, making Stocker suitable for both casual investors and professional traders.

Entity Relationship (ER) Diagram



Scenario 1 – Secure User Authentication

Stocker implements encrypted password storage, secure session management, and role-based access controls. These measures safeguard trading accounts and ensure that sensitive financial data remains protected.

Scenario 2 – Real-Time Trade Execution

Stocker's backend is optimized for **low-latency performance**, enabling instant buy/sell order processing. By leveraging AWS infrastructure and DynamoDB's fast data access, users can respond swiftly to market fluctuations and maximize trading opportunities.

Scenario 3 – Portfolio Tracking

Stocker provides an intuitive dashboard for monitoring holdings, performance trends, and transaction history. Real-time data retrieval from DynamoDB ensures that investors always have up-to-date insights into their portfolio health.

Technical Architecture

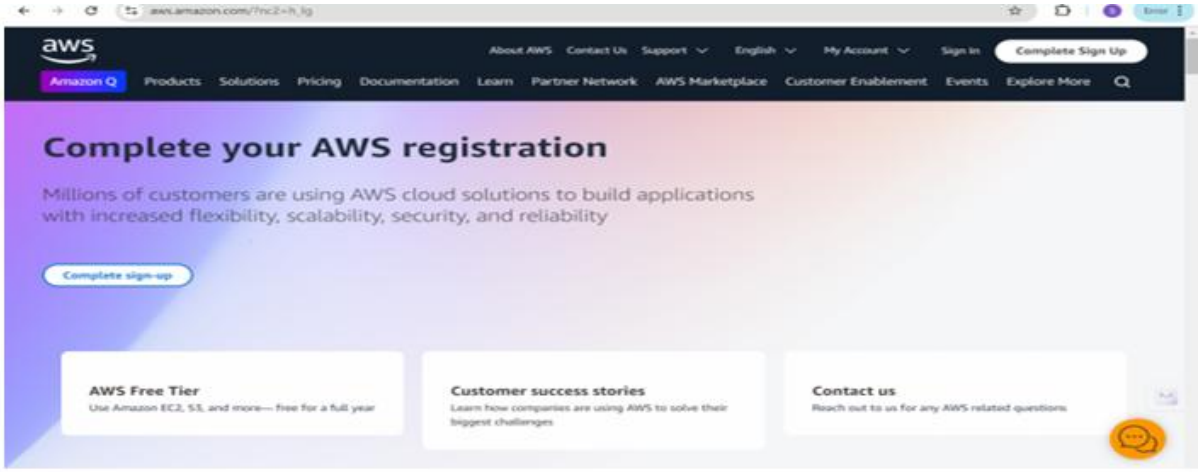
- **Flask** – Backend framework for trading logic and API handling.
- **AWS EC2** – Cloud server hosting the application with scalability.
- **DynamoDB** – NoSQL database for storing user profiles, trades, and portfolios.
- **Gunicorn & Nginx** – Production deployment stack for reliability.
- **AWS CloudWatch** – Monitoring performance, latency, and logs.

Project Implementation Steps

1. **Environment Setup & Cloud Configuration**
    - Install Flask and boto3.
    - Configure AWS credentials and EC2 hosting.
2. **Designing the Cloud Database**
    - Create DynamoDB tables for Users, Orders, and Portfolios.
    - Define keys such as UserID, OrderID, and AssetID.
3. **Building the Flask Application**
    - Secure user registration and login.
    - Real-time buy/sell order execution.
    - Portfolio dashboard and transaction history.
4. **Secure Authentication**
    - Implement encrypted password handling and role-based access.
5. **Real-Time Trade Execution**
    - Optimize backend for low latency and fast order processing.
6. **Portfolio Tracking**
    - Build dashboards with live data integration.
7. **Deployment on EC2**
    - Deploy using Gunicorn and Nginx for production.
8. **Monitoring & Optimization**
    - Use CloudWatch for performance monitoring.
    - Apply scalability and security best practices.

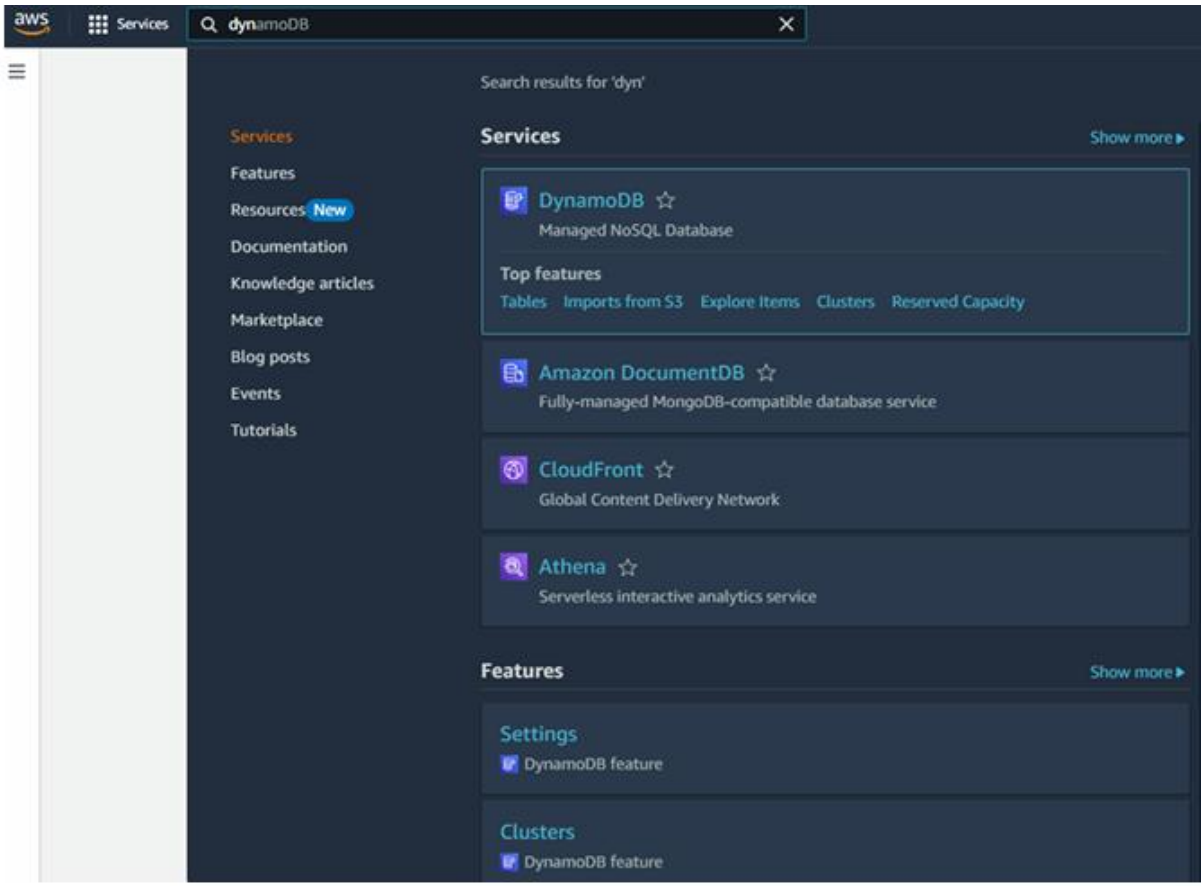
Milestone 1: AWS Account Setup and Login

● Activity 1.1: Set up an AWS account

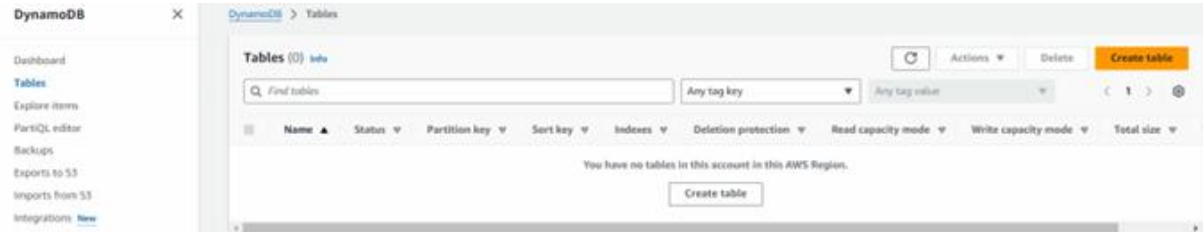


Milestone 2: DynamoDB Database Creation and Setup

- **Activity 2.1:Navigate to the DynamoDB**

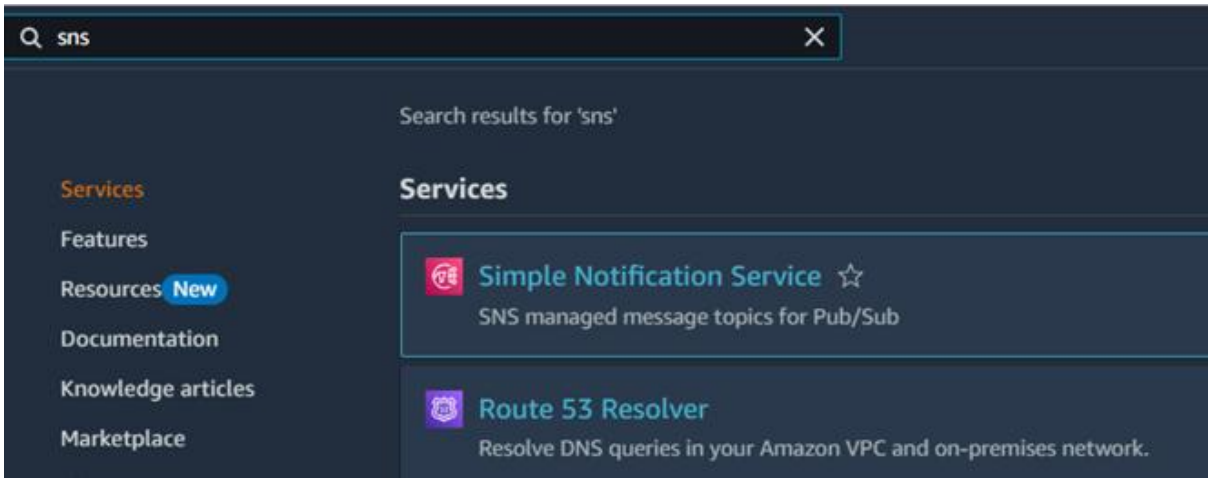


- **Activity 2.2:Create DynamoDB tables for storing registration details**

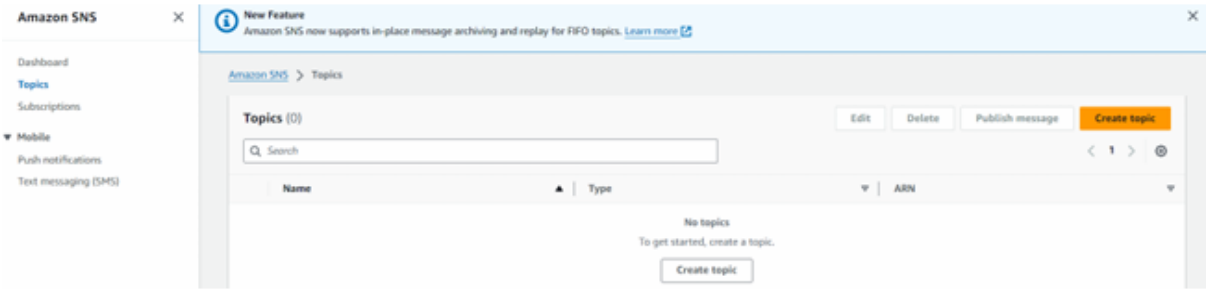


Milestone 3: SNS Notification Setup

Activity 3.1: Create an SNS Topic named Stocker for sending notifications.

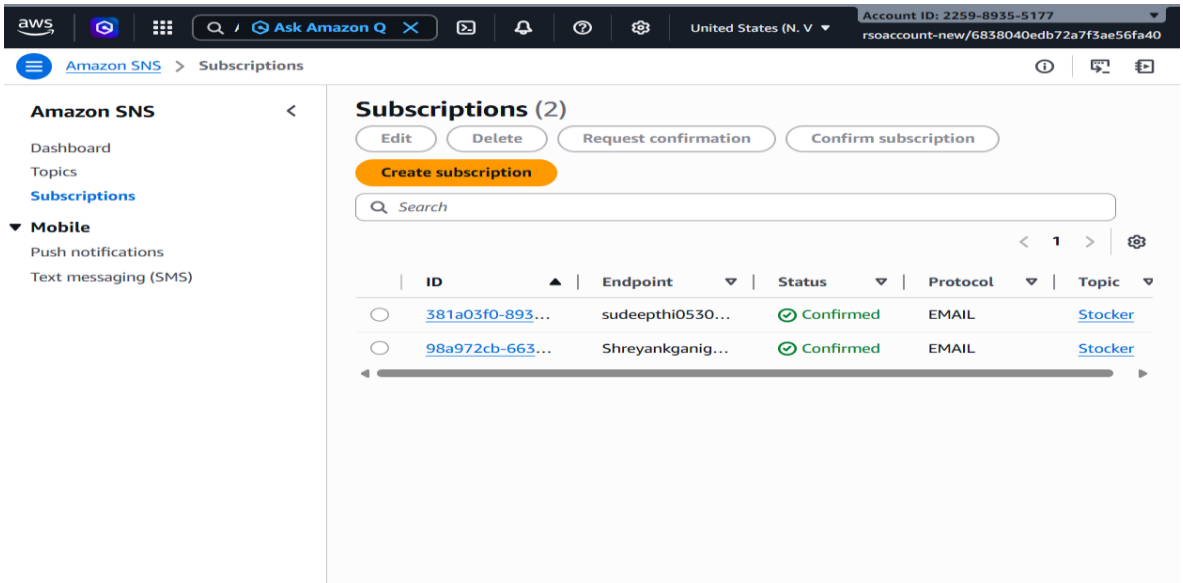


○ **Click on Create Topic and choose a name for the topic.**



○ **Configure the SNS topic and note down the Topic ARN.**

○ **After subscription request for the mail confirmation**



○ **Navigate to the subscribed Email account and Click on the confirm subscription in the AWS Notification- Subscription Confirmation mail.**

You have chosen to subscribe to the topic:

arn:aws:sns:us-east-1:225989355177:Stocker

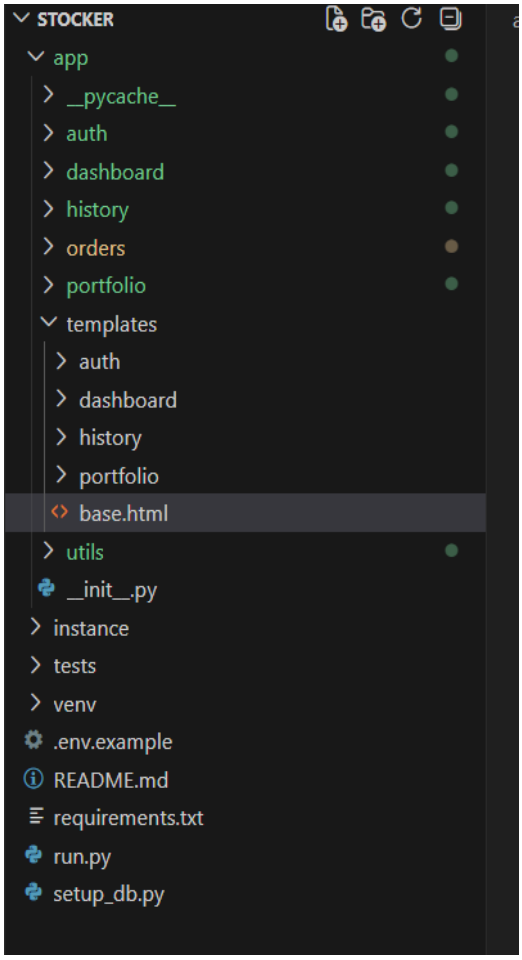
To confirm this subscription, click or visit the link below (If this was in error no action is necessary):

Confirm subscription

Please do not reply directly to this email. If you wish to remove yourself from receiving all future SNS subscription confirmation requests please send an email to sns-opt-out

Milestone 4:Backend Development and Application Setup

● Activity 4.1: Develop the backend using Flask



Description of the code :

● Flask App Initialization

Import Libraries:

```
import os
import boto3
from dotenv import load_dotenv
```

```python
import os
import boto3
from flask import Flask
from dotenv import load_dotenv
```

DynamoDB Connection:

```python
# We can initialize a global resource or rely on current_app.dynamodb
# For standalone scripts, we use this:
def get_dynamodb_client():
    return boto3.client(
        'dynamodb',
        region_name=os.environ.get('AWS_DEFAULT_REGION', 'us-east-1'),
        aws_access_key_id=os.environ.get('AWS_ACCESS_KEY_ID', 'test'),
        aws_secret_access_key=os.environ.get('AWS_SECRET_ACCESS_KEY', 'test')
    )

def create_table(dynamodb, table_name, key_schema, attribute_definitions, global_secondary_ind
    try:
        dynamodb.describe_table(TableName=table_name)
        print(f"Table '{table_name}' already exists.")
    except dynamodb.exceptions.ResourceNotFoundException:
        print(f"Creating table '{table_name}'...")
        params = {
            'TableName': table_name,
            'KeySchema': key_schema,
            'AttributeDefinitions': attribute_definitions,
            'BillingMode': 'PAY_PER_REQUEST'
        }
```

• Routes for Web Pages

```python
import functools
from flask import Blueprint, flash, g, redirect, render_template, request, session, url_for
from werkzeug.security import check_password_hash, generate_password_hash
from app.utils.dynamodb import get_user_by_email, get_user_by_id, create_user
from . import bp

@bp.route('/register', methods=('GET', 'POST'))
def register():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        error = None

        if not email:
            error = 'Email is required.'
        elif not password:
            error = 'Password is required.'

        if error is None:
            existing = get_user_by_email(email)
            if existing:
                error = f"User {email} is already registered."
            else:
                create_user(email, generate_password_hash(password))
                return redirect(url_for('auth.login'))
```

```python

        flash(error)

    return render_template('auth/register.html')

@bp.route('/login', methods=('GET', 'POST'))
def login():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        error = None
        user = get_user_by_email(email)

        if user is None:
            error = 'Incorrect email.'
        elif not check_password_hash(user['password_hash'], password):
            error = 'Incorrect password.'

        if error is None:
            session.clear()
            session['user_id'] = user['user_id']
            # We'll need to create the dashboard blueprint next
            return redirect(url_for('dashboard.index'))

        flash(error)
```

```python
    return render_template('auth/login.html')

@bp.before_app_request
def load_logged_in_user():
    user_id = session.get('user_id')

    if user_id is None:
        g.user = None
    else:
        g.user = get_user_by_id(user_id)

@bp.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('auth.login'))

def login_required(view):
    @functools.wraps(view)
    def wrapped_view(**kwargs):
        if g.user is None:
            return redirect(url_for('auth.login'))
        return view(**kwargs)
    return wrapped_view
```

Dashboard route:

```python
from flask import Blueprint, render_template, g
from app.auth.routes import login_required
from app.utils.dynamodb import get_transactions
from . import bp

@bp.route('/')
@login_required
def index():
    user_id = g.user['user_id']
    transactions = get_transactions(user_id)
    return render_template('history/index.html', transactions=transactions)

```

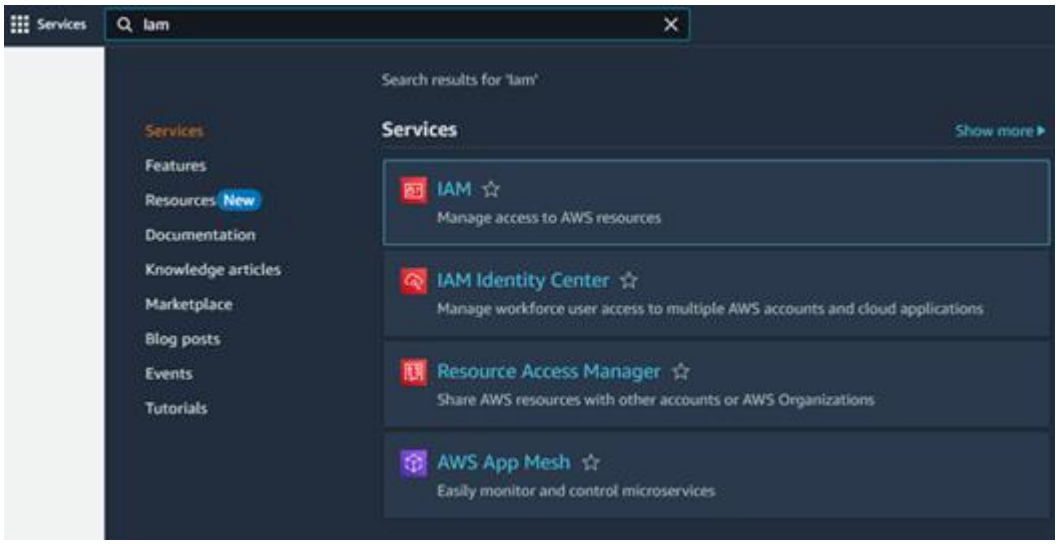
Order route

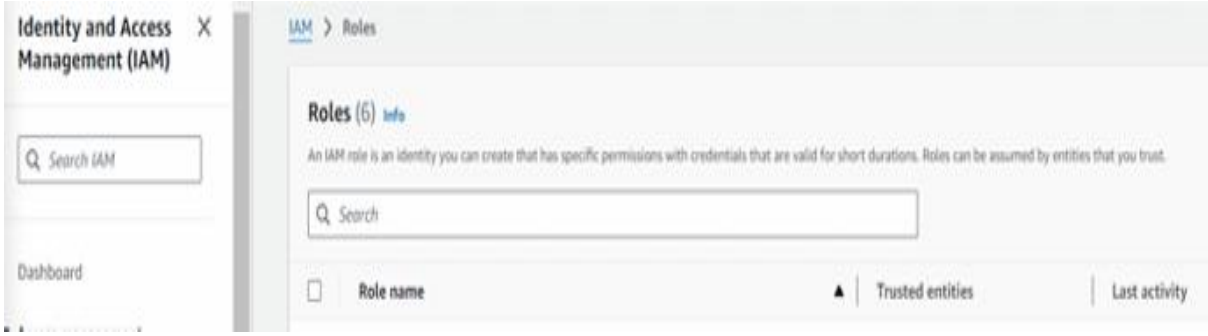
```python
from flask import Blueprint, request, redirect, url_for, flash, g
from app.auth.routes import login_required
from app.utils.dynamodb import update_portfolio, create_transaction, get_portfolio_item
from app.utils.sns import send_trade_notification
from . import bp

@bp.route('/place', methods=['POST'])
@login_required
def place():
    symbol = request.form['symbol']
    price = request.form['price']
    tx_type = request.form['type']


    try:
        quantity = int(request.form['quantity'])
    except ValueError:
        flash("Invalid quantity.")
        return redirect(url_for('dashboard.index'))

    if quantity <= 0:
        flash("Quantity must be greater than 0.")
        return redirect(url_for('dashboard.index'))

    user_id = g.user['user_id']
```

```python

    if tx_type == 'buy':
        try:
            update_portfolio(user_id, symbol, quantity)
            create_transaction(user_id, symbol, 'buy', quantity, price)
            send_trade_notification(g.user['email'], symbol, 'buy', quantity, price)
            flash(f"Successfully bought {quantity} shares of {symbol}.")
        except Exception as e:
            flash(f"Error buying stock: {str(e)}")

    elif tx_type == 'sell':
        # Check if they have enough
        item = get_portfolio_item(user_id, symbol)
        current_qty = int(item['quantity']) if item else 0
        if current_qty < quantity:
            flash(f"Not enough shares to sell. You have {current_qty}.")
        else:
            try:
                update_portfolio(user_id, symbol, -quantity)
                create_transaction(user_id, symbol, 'sell', quantity, price)
                send_trade_notification(g.user['email'], symbol, 'sell', quantity, price)
                flash(f"Successfully sold {quantity} shares of {symbol}.")
            except Exception as e:
                flash(f"Error selling stock: {str(e)}")
```

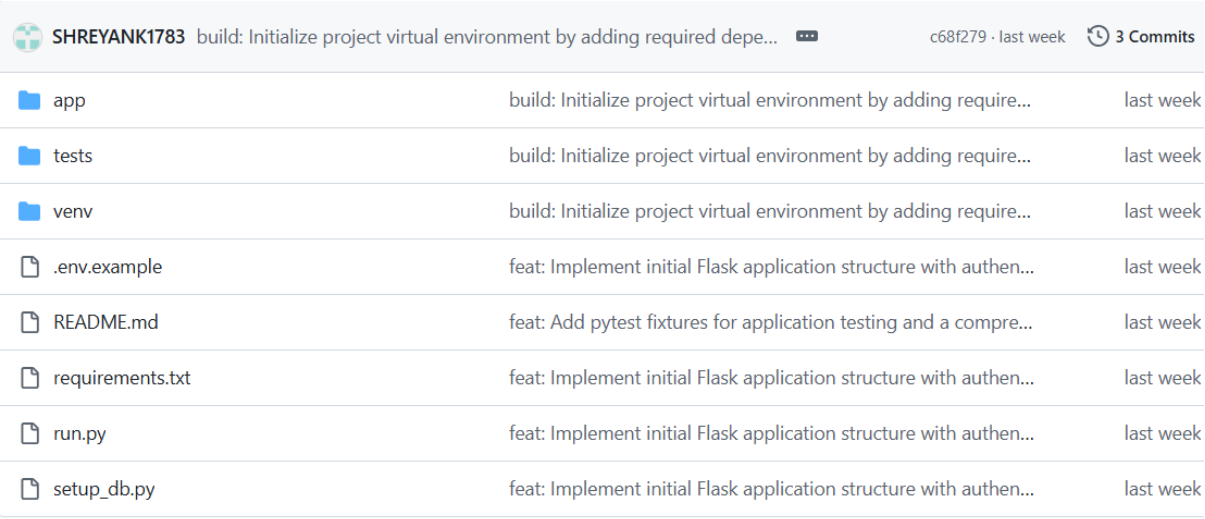
Milestone 5: IAM Role Setup

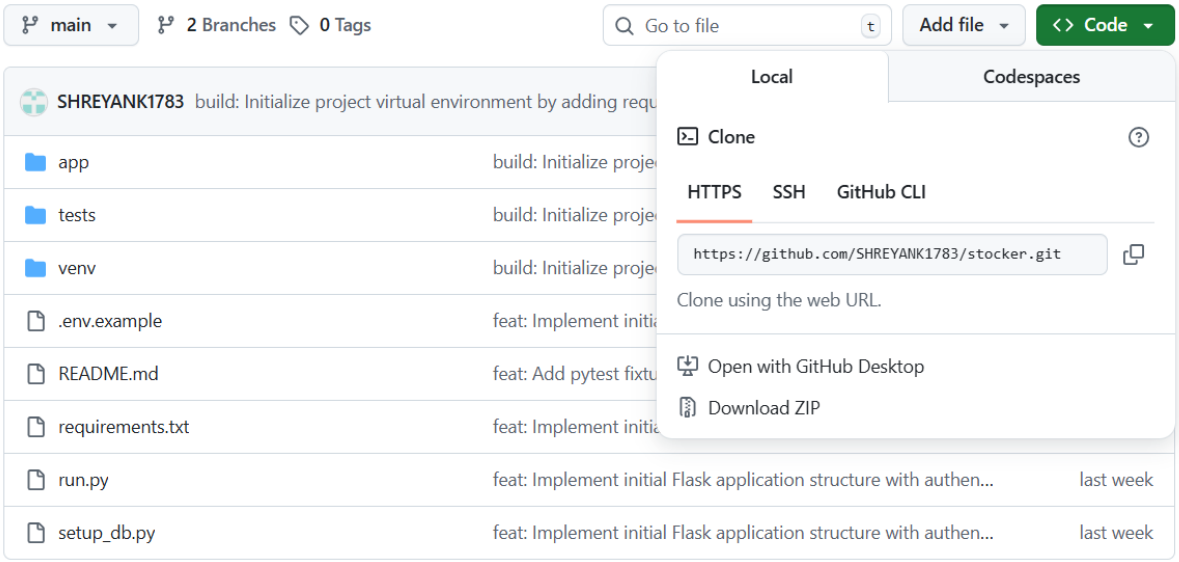
● Activity 5.1:Create IAM Role.



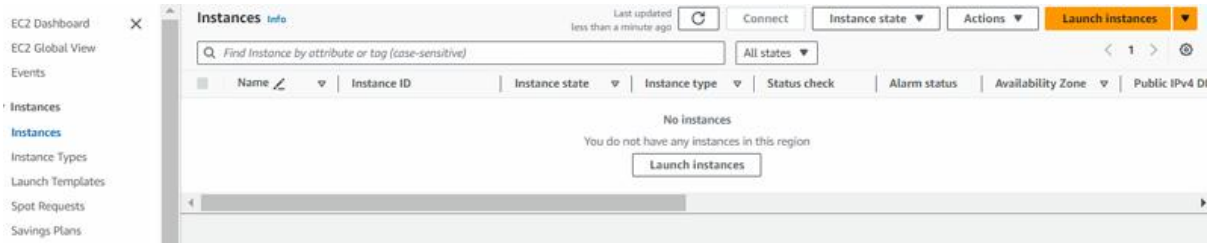


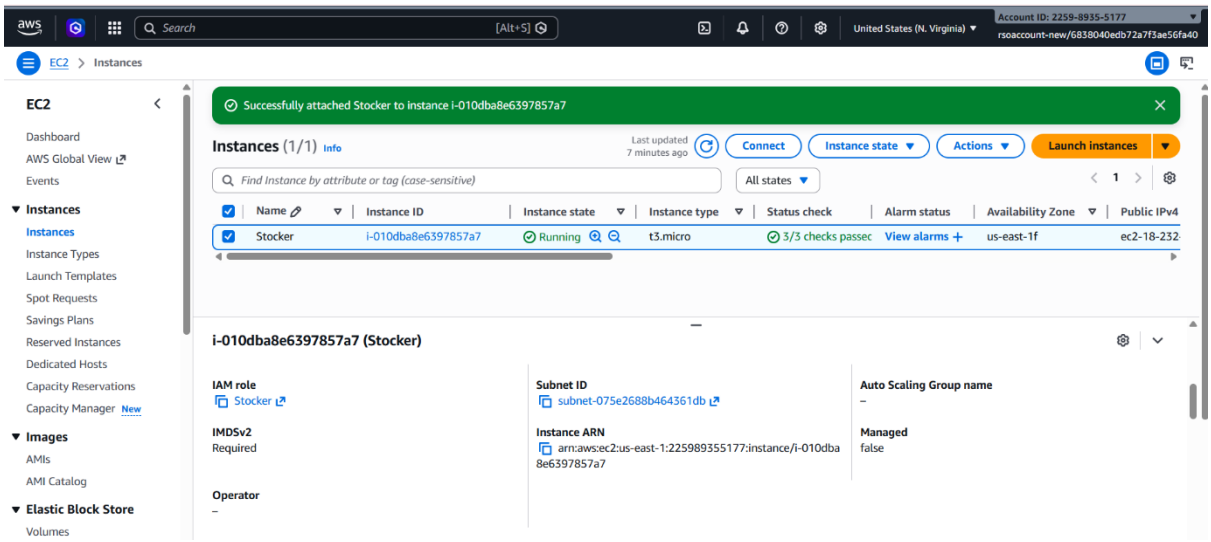
Milestone 6: EC2 Instance Setup

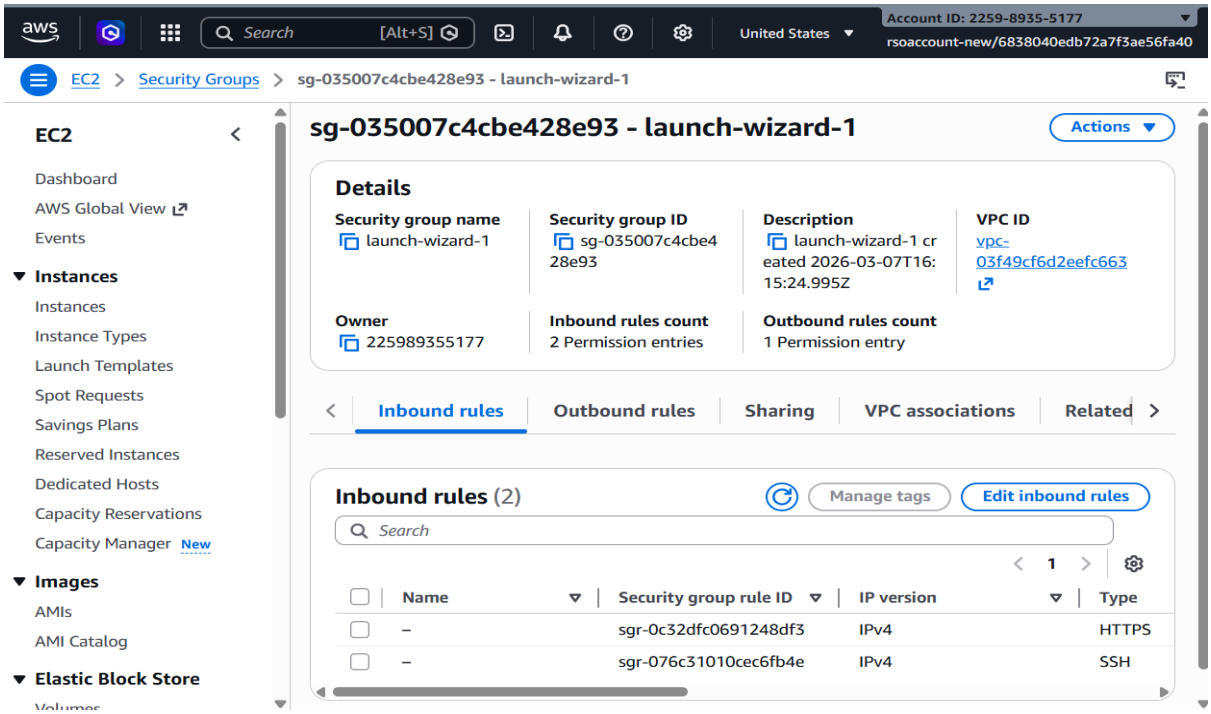




- **Activity 6.1: Launch an EC2 instance to host the Flask application.**
  - **Launch EC2 Instance**







Milestone 7: Deployment on EC2

Activity 7.1: Install Software on the EC2 Instance

Update and install Python + pip

sudo apt update

sudo apt install python3-pip -y

sudo apt install git -y

sudo apt install python3-venv -y

python3 -m venv venv

source venv/bin/activate

Activity 7.2:Clone Your Flask Project from GitHub

Clone your project repository from GitHub into the EC2 instance using Git.

Run: 'git clone https://github.com/SHREYANK1783/stocker.git'

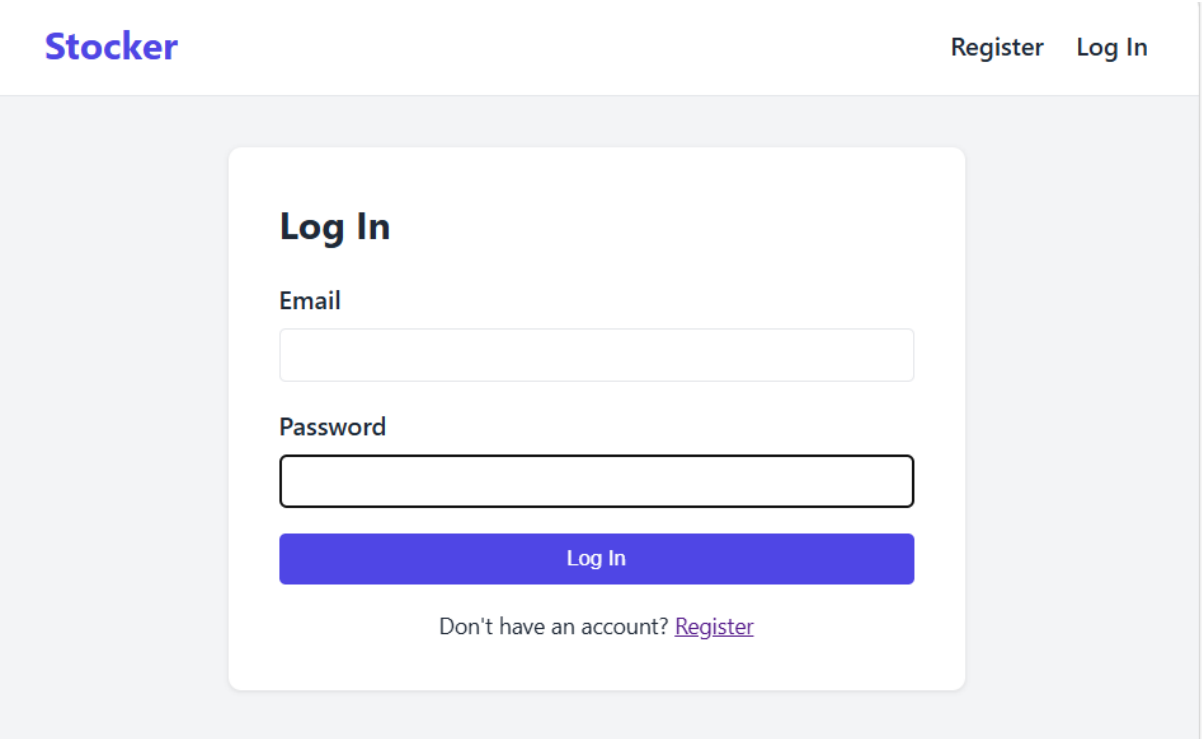
Access the website through:

PublicIPs: http://127.0.0.1:5000

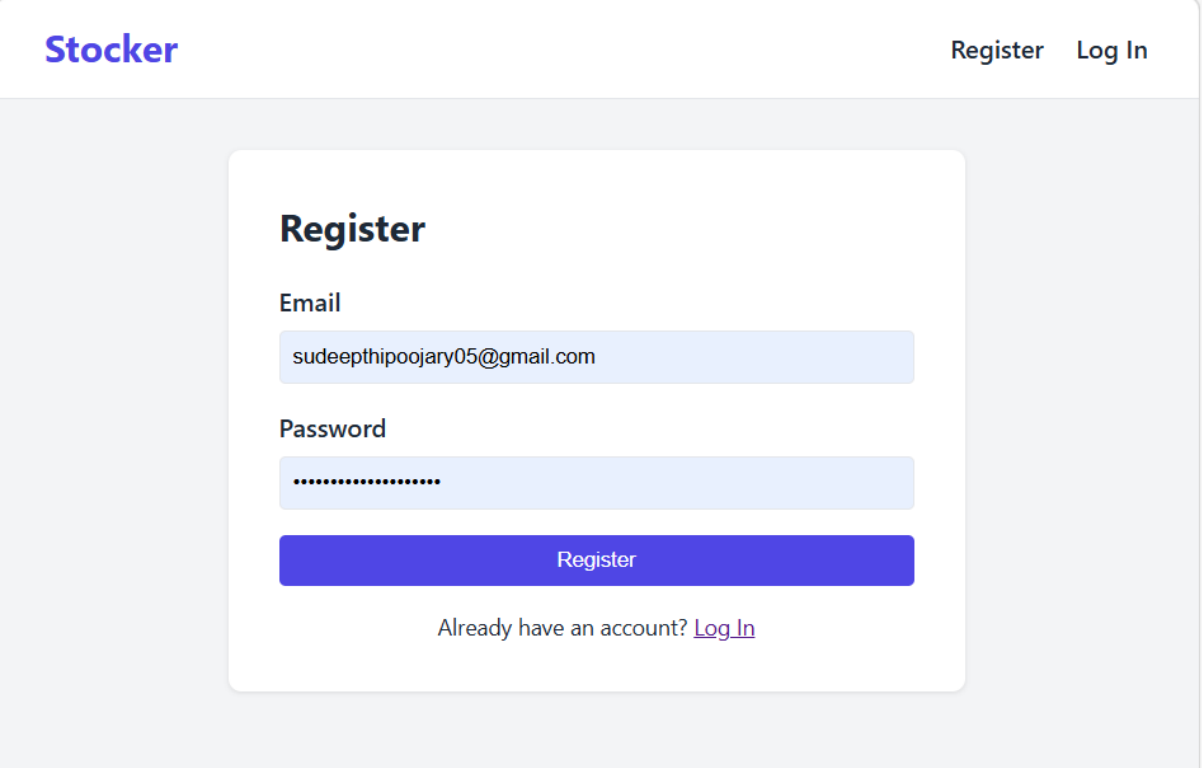
Milestone 8: Testing and Deployment

● Activity 8.1: Conduct functional testing to verify user registration, login, book requests, and notifications.

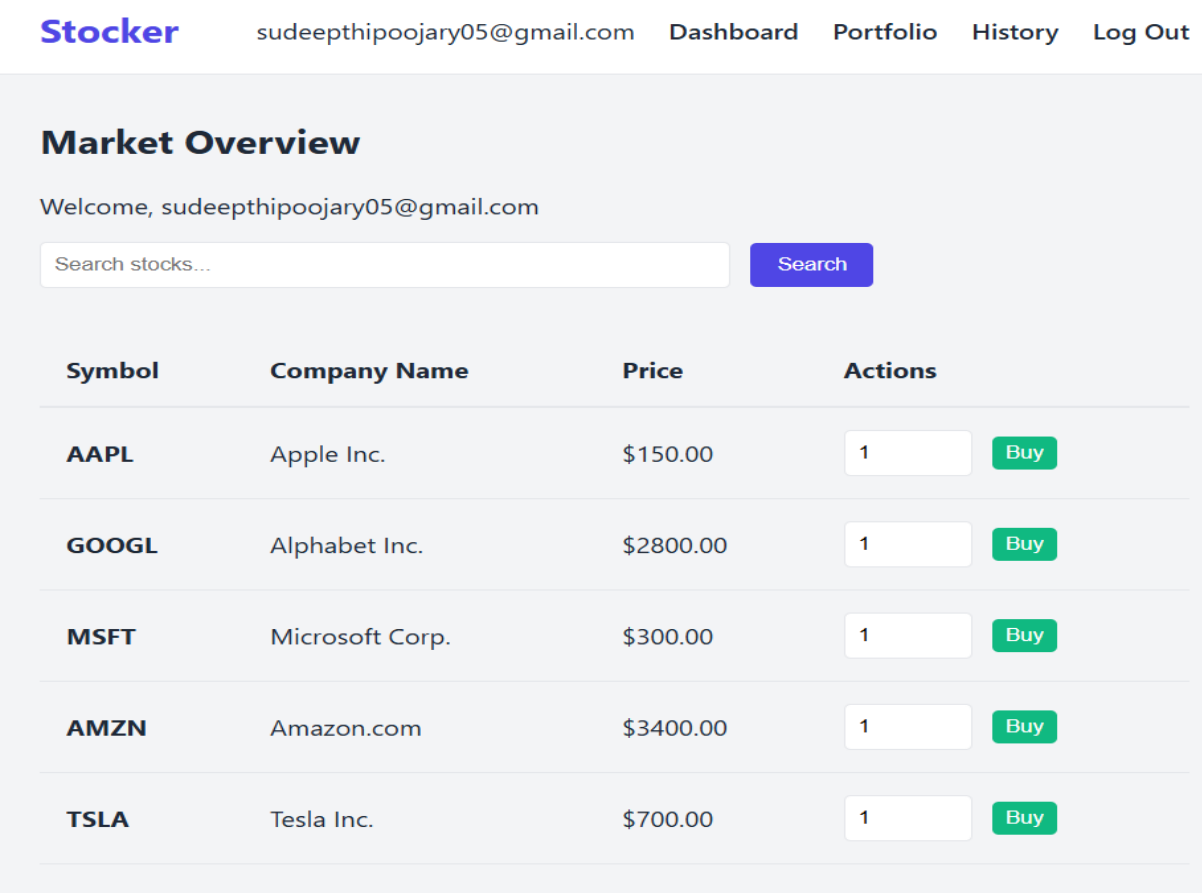
Login Page:



Register Page:



Dashboard page:



Portfolio page

Stocker sudeepthipoojary05@gmail.com Dashboard Portfolio History Log Out

Successfully bought 1 shares of AMZN.

Portfolio Summary

Total Value

$3550.00

Symbol	Shares	Current Price	Total Value	Actions
AAPL	1	$150.00	$150.00	1 Sell
AMZN	1	$3400.00	$3400.00	1 Sell

History page:

Stocker sudeepthipoojary05@gmail.com Dashboard Portfolio History Log Out

Your Recent Transactions

Date	Type	Symbol	Quantity	Price	Total Amount
2026-03-12 08:33:57	SELL	AAPL	1	$150.00	+$150.00
2026-03-12 08:33:40	BUY	AMZN	1	$3400.00	-$3400.00
2026-03-12 08:33:15	BUY	AAPL	1	$150.00	-$150.00